DATA STRUCTURES & ALGORITHMS

Full Notes for B.Tech CSE

Concepts • Complexity • Algorithms • Java Implementations • Interview Patterns

Coverage: foundations through linked lists, stacks, queues, trees, heaps, hashing, graphs, sorting, searching, recursion, greedy algorithms, dynamic programming, backtracking, and interview problem-solving patterns.

How to use: Learn the idea → write the algorithm → code it without looking → analyse time/space → solve 3–5 problems.

1. DSA Foundations

What is a Data Structure?

A data structure is a method of organising and storing data so operations such as access, insertion, deletion, searching and updating can be performed efficiently.

Types

• Linear: array, linked list, stack, queue, deque.

• Non-linear: tree, heap, graph.

• Hash-based: hash table / hash map.

• Static structures have a fixed-size allocation; dynamic structures can grow or shrink.

Algorithm

An algorithm is a finite sequence of unambiguous steps that transforms input into output. A good algorithm is correct, terminates, and uses reasonable time and memory.

Complexity

Time complexity describes how running time grows with input size n. Space complexity describes additional memory usage.

• O(1): constant

• O(log n): logarithmic

• O(n): linear

• O(n log n): common efficient sorting

• O(n^2): quadratic

• O(2■), O(n!): exponential/factorial; usually expensive

Asymptotic notation

• Big-O: upper-bound growth.

• Big-Ω: lower-bound growth.

• Big-Θ: tight asymptotic bound.

Amortized analysis

An operation can occasionally be expensive while the average cost over a sequence remains small. Dynamic-array append is a standard example.

2. Arrays

An array stores elements in contiguous memory and provides O(1) indexed access. In Java, arrays have fixed length.

Core operations

• Access: O(1)

• Search: O(n) unsorted; O(log n) in a sorted array using binary search.

• Insert/delete at middle: O(n) because elements may need shifting.

• Traverse: O(n).

Common patterns

• Two pointers: maintain left/right indices.

• Sliding window: maintain a moving range.

• Prefix sum: precompute cumulative sums for fast range-sum queries.

• Frequency counting: store occurrence counts when the value range permits.

```
int[] a = {10, 20, 30, 40};
for (int x : a) System.out.println(x);
```

Prefix sum

```
int[] pref = new int[a.length];
pref[0] = a[0];
for (int i = 1; i < a.length; i++)
pref[i] = pref[i-1] + a[i];
// sum l..r = pref[r] - (l == 0 ? 0 : pref[l-1])
```

3. Strings

A Java String is immutable. Repeated concatenation in a loop can create many objects; StringBuilder is preferable for frequent modifications.

• Common tasks: character frequency, palindrome check, substring search, anagram check, two-pointer processing.

• ASCII lowercase frequency can use int[26].

• Always clarify whether comparison is case-sensitive and whether spaces/punctuation count.

```
boolean palindrome(String s) {
int l = 0, r = s.length() - 1;
while (l < r) {
if (s.charAt(l++) != s.charAt(r--)) return false;
}
return true;
}
```

4. Recursion

Recursion solves a problem by calling the same function on a smaller subproblem. Every recursive solution needs a base case and progress toward it.

```
int factorial(int n) {
if (n <= 1) return 1;
return n * factorial(n - 1);
}
```

Recursion uses the call stack. A recursion depth of O(n) can therefore require O(n) stack space.

Backtracking

Backtracking explores choices, recursively builds a partial solution, and undoes the choice when returning. It is useful for subsets, permutations, combinations and constraint problems.

5. Linked Lists

A linked list consists of nodes connected by references. Unlike arrays, nodes need not be contiguous in memory.

Singly linked list

```
class Node {
int data;
Node next;
Node(int data) { this.data = data; }
}
```

• Access/search: O(n).

• Insert at head: O(1).

• Delete head: O(1).

• Insert after a known node: O(1).

• Finding the node first may still cost O(n).

Reverse a linked list

```
Node reverse(Node head) {
Node prev = null, cur = head;
while (cur != null) {
Node next = cur.next;
cur.next = prev;
prev = cur;
cur = next;
}
return prev;
}
```

Fast and slow pointers

Use two pointers moving at different speeds to find a middle node or detect a cycle. Floyd's cycle detection uses O(1) extra space.

Doubly linked list

Each node has next and previous references. Deletion can be easier when a node reference is already known, at the cost of extra memory.

6. Stack

A stack follows LIFO: last in, first out.

• Operations: push, pop, peek/top, isEmpty.

• Typical complexity: O(1) per operation.

• Uses: function calls, undo, parentheses matching, DFS, monotonic-stack problems.

```
Deque<Integer> st = new ArrayDeque<>();
st.push(10);
st.push(20);
int top = st.peek();
int removed = st.pop();
```

Prefer ArrayDeque for a stack in modern Java rather than the legacy Stack class.

7. Queue, Deque and Priority Queue

A queue follows FIFO: first in, first out.

• Queue operations: offer/enqueue, poll/dequeue, peek.

• Circular queues reuse freed positions in a fixed-size buffer.

• Deque supports insertion/removal at both ends.

• PriorityQueue removes the highest-priority element according to its ordering; Java's default is a min-heap.

```
Queue<Integer> q = new ArrayDeque<>();
q.offer(10);
q.offer(20);
int x = q.poll();

PriorityQueue<Integer> minHeap = new PriorityQueue<>();
```

```
minHeap.offer(5);
minHeap.offer(2);
```

8. Hashing

Hashing maps a key to a bucket using a hash function. Average-case lookup, insertion and deletion in a well-distributed hash table are approximately O(1), although worst-case behaviour can be O(n).

• HashMap: key-value mapping.

• HashSet: unique elements.

• Use hashing for frequency maps, duplicate detection, pair-sum problems and fast membership tests.

• Collisions occur when multiple keys map to the same bucket; implementations resolve them internally.

```
HashMap<Integer, Integer> freq = new HashMap<>();
for (int x : a) freq.put(x, freq.getOrDefault(x, 0) + 1);
```

9. Searching

Linear search

Check elements sequentially. Time O(n), space O(1).

Binary search

Binary search requires a sorted search space and repeatedly halves the range. Time O(log n), space O(1) iteratively.

```
int binarySearch(int[] a, int target) {
int lo = 0, hi = a.length - 1;
while (lo <= hi) {
int mid = lo + (hi - lo) / 2;
if (a[mid] == target) return mid;
if (a[mid] < target) lo = mid + 1;
else hi = mid - 1;
}
return -1;
}
```

Binary search on answer

When a problem asks for the minimum/maximum feasible value and feasibility is monotonic, binary-search the answer rather than an array.

10. Sorting

Algorithm	Best	Average	Worst	Extra space
Bubble	O(n) *	O(n²)	O(n²)	O(1)
Selection	O(n²)	O(n²)	O(n²)	O(1)
Insertion	O(n) *	O(n²)	O(n²)	O(1)
Merge	O(n log n)	O(n log n)	O(n log n)	O(n)
Quick	O(n log n)	O(n log n)	O(n²)	O(log n) avg stack
Heap	O(n log n)	O(n log n)	O(n log n)	O(1)

* Best case assumes the appropriate optimisation, such as an already-sorted check for bubble sort or insertion sort.

Merge sort

Divide the array into halves, recursively sort both halves, then merge. It guarantees O(n log n) time and is stable.

Quick sort

Choose a pivot, partition elements around it, then recursively sort the partitions. Good pivot selection gives expected O(n log n); poor partitions can produce $O(n^2)$.

11. Trees

A tree is a hierarchical structure. A binary tree has at most two children per node.

• Root: top node. Leaf: node with no children.

• Height: longest downward path measured by edges (conventions can vary).

• Depth: distance from root.

• Balanced trees keep heights relatively small.

Traversals

• Preorder: Root → Left → Right.

• Inorder: Left → Root → Right.

• Postorder: Left → Right → Root.

• Level order: breadth-first by levels.

```
void inorder(Node root) {
if (root == null) return;
inorder(root.left);
System.out.print(root.data + " ");
inorder(root.right);
}
```

Binary Search Tree

For a BST, keys in the left subtree are smaller and keys in the right subtree are larger under the chosen ordering rule. Search/insert/delete are O(h), where h is tree height. Balanced: O(log n); skewed: O(n).

AVL / balanced BST idea

Self-balancing BSTs maintain height near logarithmic by rotations after updates. This prevents repeated operations from degrading to linear height.

12. Heap

A binary heap is a complete binary tree represented efficiently in an array. In a min-heap, each parent is ≤ its children; in a max-heap, each parent is ≥ its children.

• Peek root: O(1).

• Insert: O(log n).

• Delete root: O(log n).

• Build heap from an array: O(n).

• Heap sort: O(n log n).

Array indices (0-based): parent=(i-1)/2, left=2i+1, right=2i+2.

13. Graphs

A graph consists of vertices and edges. It may be directed/undirected and weighted/unweighted.

Representations

• Adjacency matrix: $O(V^2)$ space; fast O(1) edge lookup.

• Adjacency list: O(V+E) space; efficient for sparse graphs.

BFS

Breadth-first search uses a queue. In an unweighted graph, BFS gives shortest path distance in number of edges from the source.

DFS

Depth-first search uses recursion or an explicit stack. It is useful for connectivity, cycle detection, components and traversal.

Dijkstra

Dijkstra finds shortest paths from one source when edge weights are non-negative. With a binary heap and adjacency list, a common bound is O((V+E) log V).

Topological sort

A topological ordering exists for a directed acyclic graph (DAG). Kahn's algorithm uses indegrees and a queue; DFS can also produce an ordering.

Minimum Spanning Tree

Kruskal sorts edges and uses Disjoint Set Union (Union-Find). Prim grows a tree from a chosen starting vertex using a priority queue.

14. Disjoint Set Union

DSU maintains disjoint sets and supports find and union. Path compression plus union by rank/size gives near-constant amortized time, commonly written $O(\alpha(n))$ per operation.

```
int find(int x) {
if (parent[x] != x) parent[x] = find(parent[x]);
return parent[x];
}
void union(int a, int b) {
a = find(a); b = find(b);
if (a != b) parent[b] = a;
}
```

15. Greedy Algorithms

A greedy algorithm repeatedly makes the locally best choice. Greedy works only when the problem has the required structural properties; proving correctness matters.

• Activity selection: choose the activity with earliest finishing time.

• Fractional knapsack: choose highest value/weight first.

• Huffman coding: repeatedly combine the two least frequent nodes.

• MST algorithms are greedy.

16. Dynamic Programming

Dynamic programming (DP) is useful when subproblems overlap and an optimal solution can be composed from optimal subsolutions.

Two styles

• Memoization: top-down recursion + cache.

• Tabulation: bottom-up table.

DP checklist

• Define the state: what information uniquely describes a subproblem?

• Write the transition: how does the current state depend on smaller states?

• Set base cases.

• Choose computation order.

• Return the target state.

• Estimate states × transition cost for complexity.

```
// Fibonacci with O(n) time and O(1) space
int fib(int n) {
if (n <= 1) return n;
int a = 0, b = 1;
for (int i = 2; i <= n; i++) {
int c = a + b;
a = b; b = c;
}
return b;
}
```

Classic DP families

• 0/1 Knapsack

• Coin change

• Longest Common Subsequence

• Longest Increasing Subsequence

• Grid path problems

• Partition / subset sum

• House robber and state-transition problems

17. Backtracking

Backtracking is structured brute force with pruning. Typical template: choose → explore → undo.

```
void generate(int i, List<Integer> cur, int[] a) {
if (i == a.length) {
// process cur
return;
}
cur.add(a[i]);
generate(i + 1, cur, a);
cur.remove(cur.size() - 1);
generate(i + 1, cur, a);
}
```

The search tree can be exponential, so pruning and constraint ordering are crucial.

18. Bit Manipulation

• x & 1 checks whether x is odd.

• x << k roughly multiplies by 2^k when overflow is not an issue.

• x >> k shifts right; behaviour depends on signedness and language rules.

• x ^ x = 0 and x ^ 0 = x.

• For non-negative x, x & (x-1) clears the lowest set bit.

• Count set bits efficiently with repeated x &= x-1.

Use bit tricks only when they make the logic clearer and the constraints justify them.

19. Advanced Interview Patterns

• Two pointers: sorted arrays, pair relationships, palindromes.

• Sliding window: contiguous subarrays/substrings with a maintainable condition.

• Prefix sum: repeated range sums and subarray-sum transformations.

• Monotonic stack: next greater/smaller element, histogram-style problems.

• Monotonic queue: sliding-window maximum/minimum.

• Heap: top-k, k-way merge, scheduling, streaming median.

• Trie: prefix queries and dictionary-like string operations.

• Binary search on answer: optimise a monotonic feasibility condition.

• Graph BFS layers: minimum number of moves in unweighted state spaces.

20. Complexity Cheat Sheet

Structure / Algorithm	Typical time	Space
Array access	O(1)	O(1)
Array search	O(n)	O(1)
Linked-list search	O(n)	O(1)
Stack push/pop	O(1)	O(n) storage
Queue offer/poll	O(1)	O(n) storage
HashMap lookup	O(1) average	O(n)
BST search	O(h)	O(h) recursion
Heap insert/delete	O(log n)	O(n)
BFS / DFS	O(V+E)	O(V)
Merge sort	O(n log n)	O(n)
Binary search	O(log n)	O(1) iterative

21. Problem-Solving Workflow

• 1. Read constraints before coding.

• 2. Identify input/output and edge cases.

• 3. Start with the brute-force idea.

• 4. Find the bottleneck.

• 5. Match the bottleneck to a pattern/data structure.

• 6. Prove or reason about correctness.

• 7. Estimate time and space.

• 8. Implement cleanly.

• 9. Test empty, one-element, duplicate, sorted/reverse-sorted and boundary cases.

22. DSA Study Order

Recommended progression for a CSE student: Complexity → Arrays/Strings → Recursion → Linked Lists → Stack/Queue → Hashing → Sorting/Searching → Trees/BST → Heap → Graphs → Greedy → Backtracking → Dynamic Programming → advanced patterns.

Reality check: Reading notes is not DSA mastery. For internship-level preparation, implementation and problem solving matter far more than memorising definitions. After each topic, solve problems without copying the solution.

23. Quick Revision Checklist

• Can you explain O(1), O(log n), O(n), O(n log n), $O(n^2)$?

• Can you implement binary search from memory?

• Can you reverse a linked list and detect a cycle?

• Can you implement stack/queue operations?

• Can you use HashMap/HashSet for frequency and membership?

• Can you write all four tree traversals?

• Can you explain heap operations and PriorityQueue?

• Can you run BFS/DFS and track visited nodes?

• Can you explain why Dijkstra requires non-negative edge weights?

• Can you identify greedy vs DP vs backtracking?

• Can you derive a DP state and transition?

• Can you analyse your solution's complexity?

End of Notes

Use this as a revision/reference document. Pair every section with coding practice.